Materia: Programación IV

UNIDAD 7 · SERVER STATE Y NAVEGACIÓN

TRABAJO PRÁCTICO

CRUD Categorías + CRUD Productos

Revisor de la unidad: Prof. Franco González

OBJETIVO GENERAL

Desarrollar una aplicación Full Stack conectando un frontend React + TypeScript con un backend FastAPI. El trabajo consiste en implementar el CRUD completo de Categorías desde la interfaz de usuario, integrando el backend mediante TanStack Query, y en organizar el frontend en páginas con React Router. Los CRUD de Producto y ProductoCategoria se construyen en el backend y se demuestran mediante un archivo de REST Client.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
React	Biblioteca para construir interfaces de usuario mediante componentes reutilizables.
TypeScript	Superconjunto tipado de JavaScript que permite detectar errores en tiempo de compilación y mejora la legibilidad.
Vite	Servidor de desarrollo y herramienta de build. Con `pnpm create vite` se genera un proyecto React + TypeScript; es la alternativa recomendada a Create React App, que está discontinuado.
FastAPI	Framework Python de alto rendimiento para construir APIs REST. Permite definir endpoints, validar datos con Pydantic y exponer documentación automática en `/docs`.
Pydantic	Librería de validación de datos usada por FastAPI para definir modelos (schemas) a partir de anotaciones de tipos.
TanStack Query	Librería para el manejo del estado del servidor. Gestiona el pedido de datos, la caché, los estados de carga y error, y la revalidación de los datos cuando se invalida una consulta.
useState	Hook de React para manejar estado local de interfaz (por ejemplo, apertura del modal, categoría seleccionada para editar). El estado del servidor lo gestiona TanStack Query.

Concepto	Aplicación en el proyecto
useQuery / useMutation	Hooks de TanStack Query: `useQuery` obtiene y cachea el listado de categorías; `useMutation` ejecuta el alta, la edición y la baja. En el callback `onSuccess` de cada mutación se invalida la consulta del listado con `queryClient.invalidateQueries`, lo que provoca que se vuelva a pedir.
React Router	Librería de enrutamiento para aplicaciones de una sola página (SPA). Con `BrowserRouter`, `Routes` y `Route` se asocia cada ruta (`/categorias`, `/productos`) a una página, y con `Link` se navega entre ellas sin recargar la página.
Interface	Construcción de TypeScript que define la forma de un objeto (por ejemplo, `Categoria`, `Producto`).
Tailwind CSS	Framework de CSS utilitario. Estilos aplicados directamente en el JSX mediante clases predefinidas.
CORS	Mecanismo del navegador por el cual el servidor indica qué orígenes pueden consultarlo. Es necesario porque el frontend (`http://localhost:5173`) y el backend (`http://localhost:8000`) usan puertos distintos y, por lo tanto, son orígenes distintos.

CASO PRÁCTICO

Parte A — Backend con FastAPI

Estructura esperada del proyecto

```
fastapi-productos/
│
├── app/
│   ├── __init__.py
│   ├── main.py
│   │
│   ├── core/
│   │   ├── __init__.py
│   │   └── database.py
│   │
│   ├── categoria/
│   │   ├── __init__.py
│   │   ├── router.py
│   │   ├── model.py
│   │   ├── schema.py
│   │   └── service.py
│   │
│   └── producto/
│       ├── __init__.py
│       ├── router.py
│       ├── model.py
│       ├── schema.py
│       └── service.py
├── .env
├── .gitignore
├── productos.http
├── requirements.txt
└── README.md
```

a) Crear y activar un entorno virtual de Python, crear el archivo `requirements.txt` con las dependencias del proyecto (`fastapi[standard]` , `sqlmodel` , `psycopg2-binary` y `python-dotenv`) e instalarlas:

```
python -m venv .venv
source .venv/bin/activate   # macOS/Linux
.venv\Scripts\activate      # Windows
pip install -r requirements.txt
```

b) Definir los models y schemas. La persistencia se realiza en PostgreSQL mediante SQLModel; la conexión se configura en `app/core/database.py` a partir de la variable `DATABASE_URL` del archivo `.env` . La tabla indica los campos mínimos de cada entidad; el diagrama UML publicado junto a esta consigna (paquete Catálogo de Productos) muestra el modelo completo, y sus campos adicionales (por ejemplo, `es_principal` o los campos de auditoría) son opcionales.

Categoria	Producto	ProductoCategoria
`id: int`	`id: int`	`producto_id: int`
`nombre: str`	`nombre: str`	`categoria_id: int`
`descripcion: str`	`descripcion: str`	
	`precio_base: float`	
	`imagenes_url: list[str]`	
	`disponible: bool`	

El precio base no puede ser negativo (por ejemplo, `precio_base: float = Field(ge=0)`). La lista `imagenes_url` se persiste en una columna de tipo arreglo de PostgreSQL, por ejemplo con `Field(default_factory=list, sa_column=Column(ARRAY(String)))`; `Column` y `String` se importan desde `sqlalchemy`, y `ARRAY`, desde `sqlalchemy.dialects.postgresql`. Las tablas se crean al iniciar la aplicación, por ejemplo con `SQLModel.metadata.create_all(engine)` en un `lifespan` de `main.py` o en `database.py`.

c) Implementar los endpoints de Categorías en `app/categoria/router.py` con un `APIRouter` (`APIRouter(prefix="/categorias", tags=["categorias"])`) y registrar los routers en `app/main.py`. En `main.py` se habilita CORS para que el frontend pueda consumir la API:

(Código de ejemplo)

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.categoria.router import router as categoria_router
from app.producto.router import router as producto_router

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:5173'],
    allow_methods=['*'],
    allow_headers=['*'],
)

app.include_router(categoria_router)
app.include_router(producto_router)
```

Endpoints requeridos: `GET /categorias/` (listado), `GET /categorias/{id}` (detalle), `POST /categorias/` (alta, responde 201), `PUT /categorias/{id}` (edición) y `DELETE /categorias/{id}` (baja, responde 204). Si el `id` no existe, se responde 404.

d) Implementar también los endpoints de Producto, con la misma estructura que los de Categorías, y los de la relación ProductoCategoria: `POST /productos/{id}/categorias` (asigna al producto la categoría indicada en el cuerpo con `categoria_id`), `GET /productos/{id}/categorias` (lista sus categorías) y `DELETE /productos/{id}/categorias/{categoria_id}` (quita la asociación). El modelo de la tabla de enlace ProductoCategoria se define en `app/producto/model.py`. Estos endpoints no se consumen desde el frontend: se demuestran con REST Client. El archivo `.http` se entrega junto con el proyecto.

e) Ejecutar el servidor desde la carpeta raíz del proyecto:

```
python -m fastapi dev app/main.py
```

Parte B — Frontend con React + TypeScript

a) Inicializar el proyecto e instalar TanStack Query y React Router:

```
pnpm create vite tp-categorias --template react-ts
cd tp-categorias
pnpm install
pnpm add @tanstack/react-query react-router-dom
```

b) Instalar y configurar Tailwind CSS siguiendo la guía oficial de instalación con Vite: https://tailwindcss.com/docs/installation/using-vite

c) Definir las interfaces en `src/types/categoria.ts` y `src/types/producto.ts`, con los mismos campos y tipos que los schemas del backend:

```
// src/types/categoria.ts
export interface Categoria {
  id: number;
  nombre: string;
  descripcion: string;
}

// src/types/producto.ts
export interface Producto {
  id: number;
  nombre: string;
  descripcion: string;
  precio_base: number;
  imagenes_url: string[];
  disponible: boolean;
}
```

d) Definir las funciones de acceso a la API en `src/services/categoria.ts` (GET, POST, PUT y DELETE de `/categorias`). Cada función comprueba `response.ok` y lanza un error si la respuesta no es exitosa, para que TanStack Query registre el estado de error.

e) Configurar el `QueryClient` y React Router en `src/main.tsx`: la aplicación se envuelve con `QueryClientProvider` y `BrowserRouter`.

(Código de ejemplo)

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import App from "./App";

const queryClient = new QueryClient();

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <QueryClientProvider client={queryClient}>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </QueryClientProvider>
  </StrictMode>
);
```

f) Crear los componentes en `src/components/` (`.tsx`):

- **Navbar.tsx** — barra de navegación con el nombre de la aplicación y enlaces (`Link`) a `/categorias` y `/productos`.
- **CategoriaCard.tsx** — tarjeta que muestra una categoría (nombre + descripción). Recibe props tipados con la interface `Categoria`. Incluye botones Editar y Eliminar.
- **CategoriaList.tsx** — recibe `Categoria[]` por props y renderiza las tarjetas.
- **CategoriaModal.tsx** — formulario de alta/edición dentro de un modal. Su apertura y cierre los controla `CategoriasPage` con un estado booleano (`useState`); el modal recibe ese estado y la función para cerrarlo por props. Llama a `onSubmit` al guardar.

g) En `src/pages/CategoriasPage.tsx` implementar la lógica principal del CRUD de categorías:

- `useQuery` para obtener el listado de categorías (`queryKey: ["categorias"]`) y manejar los estados de carga y error.
- `useMutation` para crear, editar y eliminar categorías. En el callback `onSuccess` de cada mutación se invalida el listado con `queryClient.invalidateQueries({ queryKey: ["categorias"] })`, lo que provoca que se vuelva a pedir (la instancia se obtiene en la página con `const queryClient = useQueryClient()`).

- `useState` únicamente para el estado del modal (abierto/cerrado) y la categoría seleccionada para edición.
- Pasar los datos y las funciones como props a los componentes hijos.

h) En `src/pages/ProductsPage.tsx` crear la página de la ruta `/productos`. Como el CRUD de productos se demuestra con REST Client y no requiere interfaz de gestión, la página puede limitarse a un título y una breve descripción de la sección.

i) En `src/App.tsx` definir las rutas y renderizar el `Navbar`. `App.tsx` no contiene estado de datos ni llamadas a la API:

(Código de ejemplo)

```
import { Routes, Route } from "react-router-dom";
import Navbar from "./components/Navbar";
import CategoriasPage from "./pages/CategoriasPage";
import ProductsPage from "./pages/ProductsPage";

function App() {
  return (
    <>
      <Navbar />
      <Routes>
        <Route path="/" element={<CategoriasPage />} />
        <Route path="/categorias" element={<CategoriasPage />} />
        <Route path="/productos" element={<ProductsPage />} />
      </Routes>
    </>
  );
}

export default App;
```

j) Verificar que el proyecto compila sin errores de TypeScript y ejecutarlo:

```
pnpm build
pnpm dev
```

El script `build` de la plantilla `react-ts` ejecuta `tsc -b` antes de `vite build`, por lo que informa los errores de tipos; `pnpm dev` solo levanta el servidor de desarrollo y no verifica tipos.

Ejemplo de pantalla del frontend

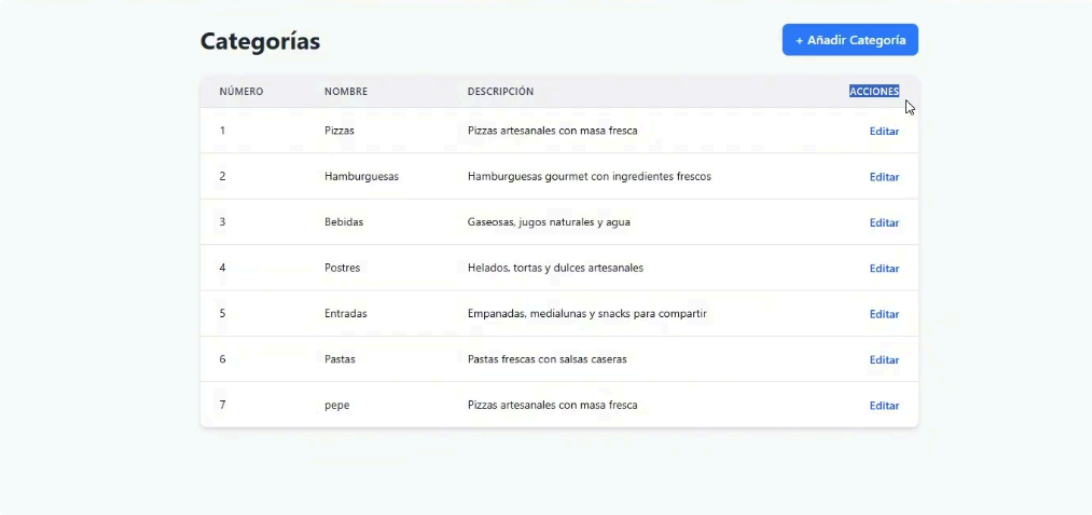


Figura 1. Listado de categorías.

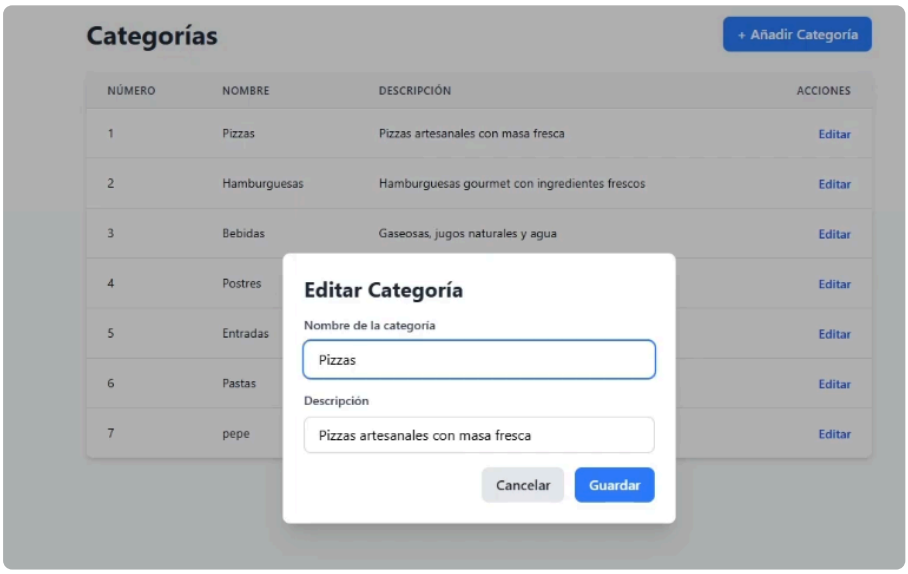


Figura 2. Modal de edición de una categoría.

Imagen de referencia: el diseño puede usar tabla o tarjetas (`CategoriaCard` puede renderizar una fila de tabla), pero cada categoría debe ofrecer las acciones Editar y Eliminar.

CONCLUSIONES ESPERADAS

Al finalizar el trabajo práctico, el estudiante debe demostrar:

- **Integración Full Stack:** El frontend React consume la API FastAPI mediante TanStack Query. El CRUD de Categorías funciona de extremo a extremo, con invalidación de la caché luego de cada operación.
- **Modal con useState:** El formulario de alta y edición está dentro de un modal controlado con estado booleano.

- **Hooks aplicados correctamente:** en `CategoriasPage.tsx`, `useQuery` y `useMutation` para el estado del servidor, y `useState` únicamente para el estado de la interfaz (modal y categoría seleccionada). Los componentes de presentación (`CategoriaList`, `CategoriaCard`) reciben datos y funciones por props y no acceden a la API; el modal puede usar estado local solo para los campos del formulario.
- **Navegación con React Router:** Las rutas `/categorias` y `/productos` muestran `CategoriasPage` y `ProductsPage`. `App.tsx` solo define las rutas y renderiza el `Navbar`.
- **Tipado TypeScript:** Interfaces `Categoria` y `Producto` definidas. Props tipados en todos los componentes `.tsx`.
- **Backend REST documentado:** Endpoints de Producto y ProductoCategoria funcionando y demostrados en el archivo `.http`.
- **CORS configurado:** El backend permite peticiones desde el origen del frontend.

RÚBRICA DE EVALUACIÓN

Criterio	Descripción	Puntos
CRUD Categorías – Frontend	Listar, crear, editar y eliminar categorías desde React usando TanStack Query (`useQuery` / `useMutation`), con invalidación de la caché luego de cada operación.	30
Modal funcional	Modal que se abre para crear y editar categorías, controlado con `useState`. Se cierra al guardar o cancelar.	15
CRUD Categorías – Backend	Endpoints FastAPI `GET /categorias/`, `GET /categorias/{id}`, `POST /categorias/`, `PUT /categorias/{id}` y `DELETE /categorias/{id}` con Pydantic y persistencia en PostgreSQL mediante SQLModel.	20
CRUD Productos y relación – REST Client	Endpoints de Productos y ProductoCategoria demostrados con REST Client (`.http`). No requiere UI.	15
Tipado TypeScript	Interfaces `Categoria` y `Producto` tipadas. Props tipados en todos los componentes.	10
Estructura y buenas prácticas	Carpetas `components/`, `pages/`, `types/` y `services/`. Rutas `/categorias` y `/productos` con React Router. Tailwind CSS aplicado. Sin errores de TypeScript al compilar.	10

Modalidad de Entrega

- Proyecto Frontend comprimido en un archivo `.zip`, sin la carpeta `node_modules`.
- Proyecto Backend comprimido en un archivo `.zip`, sin las carpetas `.venv` y `__pycache__`, con el archivo `.http` de REST Client.

RECURSOS ADICIONALES

Documentación oficial:

- React: https://react.dev/
- TypeScript: https://www.typescriptlang.org/docs/
- Tailwind CSS: https://tailwindcss.com/docs/
- Tailwind CSS con Vite: https://tailwindcss.com/docs/installation/using-vite
- Vite: https://vite.dev/guide/
- FastAPI: https://fastapi.tiangolo.com/
- FastAPI, CORS: https://fastapi.tiangolo.com/tutorial/cors/
- SQLModel: https://sqlmodel.tiangolo.com/
- TanStack Query: https://tanstack.com/query/latest/docs/framework/react/overview
- TanStack Query, invalidación desde mutaciones: https://tanstack.com/query/latest/docs/framework/react/guides/invalidations-from-mutations
- React Router: https://reactrouter.com/start/declarative/installation (en la versión 7, `react-router-dom` reexporta las mismas APIs que `react-router`; en este trabajo se usa `react-router-dom`, como en la Guía de Enrutamiento)